OMDS-ModB2-Week8-Zucker-Scharff-Thomas

March 16, 2026

1 Week 8 - Multivariate Analysis

1.1 Please run the cells of the notebook as you get to them while reading

[23]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import seaborn as sns
from sklearn import decomposition
from sklearn import linear_model
from sklearn import metrics
import matplotlib.pyplot as plt
```

2 1. Lesson: Multivariate graphs

In this lesson, we'll consider some multivariate plots: heatmaps and bubble plots. We'll also consider principal component analysis and multiple regression.

2.0.1 Heatmaps

In this plot, we use a heatmap to count the total traffic tickets given to citizens. A heatmap requires the x and y axes to be categorical, while the color involves a third, numerical variable. In this example, each row of the DataFrame represents a traffic ticket given to one person in a particular city and year. The data are given as three columns (city, year, and ticket), so we must use a pivot_table to transform it into a two-dimensional grid. The cmap is set to "Blues" in the first example, meaning that the heatmap uses only blue as its color. However, we could also use a multicolor map like "viridis" in the second example. I personally think "Blues" is easier to read - what do you think? Here is a reference guide to the color maps:

https://matplotlib.org/stable/gallery/color/colormap_reference.html

[2]:
```
np.random.seed(0)
num_samples = 100
city_series = np.random.choice(["New York", "Boston", "Atlanta", "Washington, D.
 ↪ C."], size = num_samples)
year_series = np.random.choice([2018, 2019, 2020], size = num_samples)
ticket_series = np.random.randint(50, 150, size = num_samples) # Traffic␣
 ↪tickets can be from $50 to $150
```

```
df = pd.DataFrame({"city": city_series, "year": year_series, "ticket":␣
↪ticket_series})
pivot = df.pivot_table(index = ["city"], columns = ["year"], values = "ticket",␣
↪aggfunc = "sum")
```

[3]:
```
df.iloc[0:5] # first five rows of a long DataFrame
```

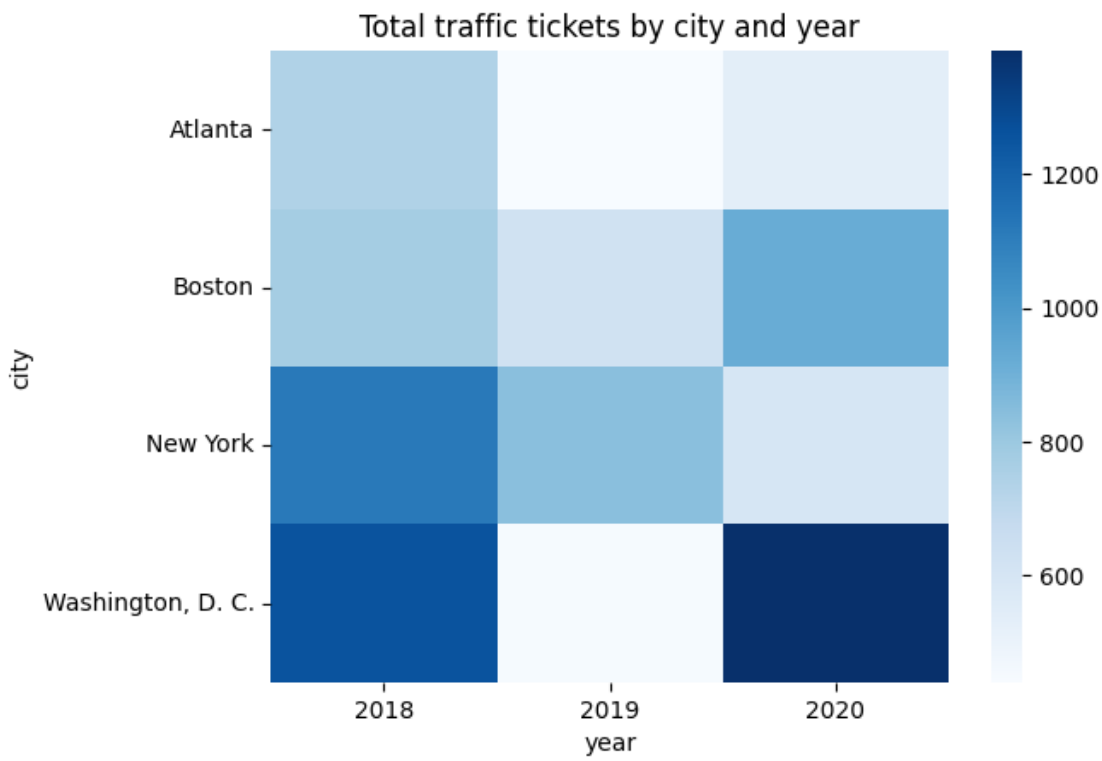
[3]:
```
                city  year  ticket
0           New York  2019      79
1  Washington, D. C.  2020      71
2             Boston  2018      75
3           New York  2018     130
4  Washington, D. C.  2020     110
```

[4]:
```
pivot # 2 by 2 grid showing total ticket amounts
```

[4]:
```
year               2018  2019  2020
city
Atlanta             742   440   538
Boston              775   628   922
New York           1122   841   596
Washington, D. C.  1258   451  1385
```

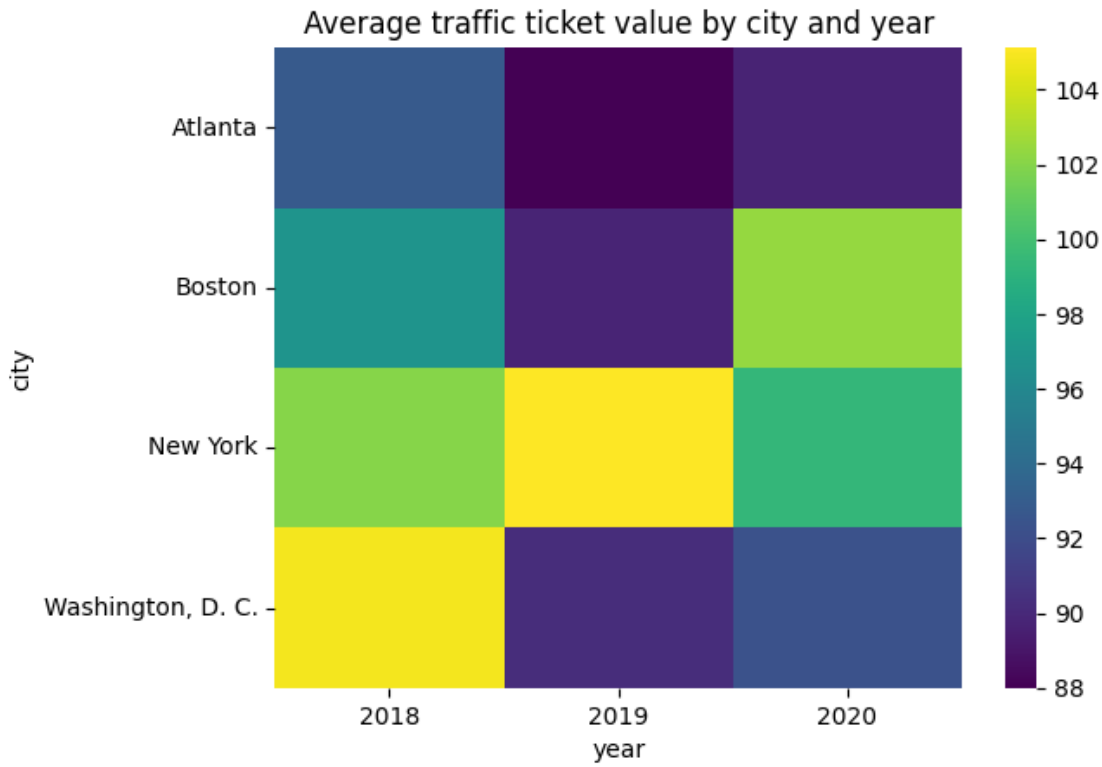
[5]:
```
ax = sns.heatmap(pivot, cmap = "Blues")
ax.set_title("Total traffic tickets by city and year")
```

[5]:
```
Text(0.5, 1.0, 'Total traffic tickets by city and year')
```



[6]:
```
pivot2 = df.pivot_table(index = ["city"], columns = ["year"], values =␣
↪"ticket", aggfunc = "mean") # note mean, not sum this time
ax2 = sns.heatmap(pivot2, cmap = "viridis") # do you think this is a better or␣
↪worse scale than Blues?
ax2.set_title("Average traffic ticket value by city and year")
```

[6]: Text(0.5, 1.0, 'Average traffic ticket value by city and year')



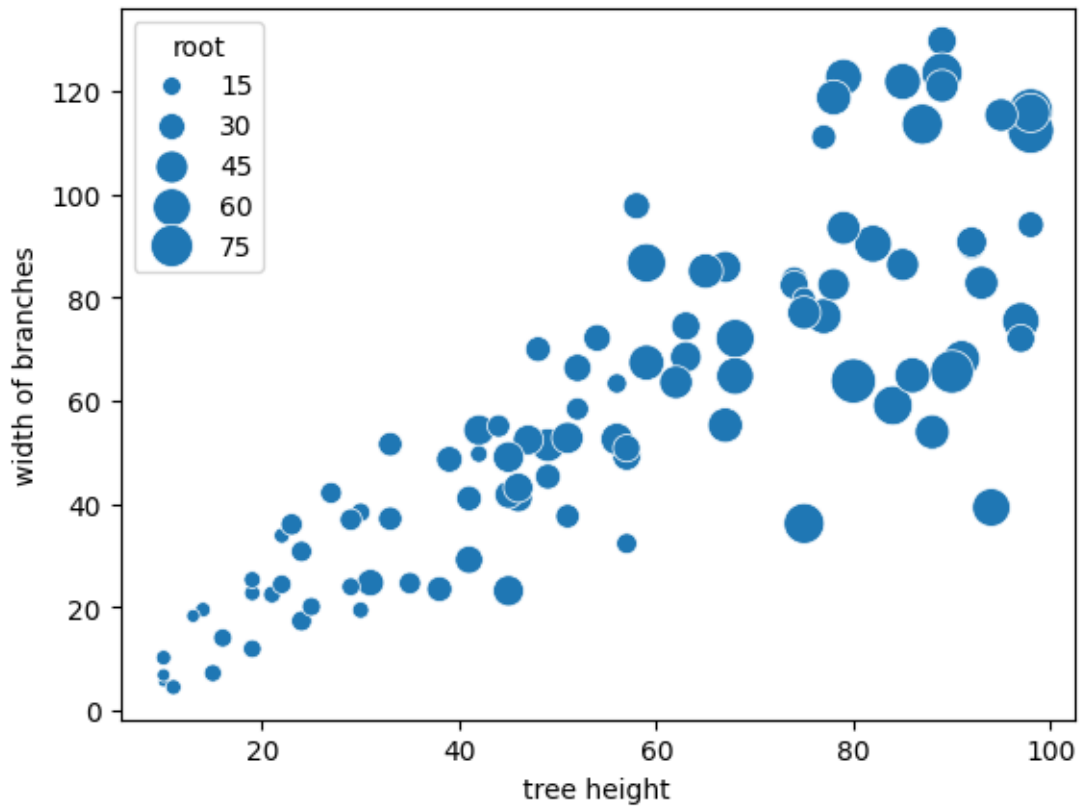
2.0.2 Bubble plots

A bubble plot is just a scatter plot, but with the dot size representing a third variable. Here, each data point shows a tree (I mean the kind of tree that is a plant and grows in the ground, not a decision tree), with its height, width of branches, and root depth. The bubble size in the bubble plot represents the root depth, while the other variables are shown on the x and y axes. From this plot, we can see that the root depth is strongly correlated with both the width of branches and the tree height.

[7]:
```
np.random.seed(0)
num_samples = 100
height_series = np.random.randint(10, 100, size = num_samples) # tree height
width_series = height_series * np.maximum(np.random.normal(scale = 0.3, loc =␣
↪1, size = num_samples), 0.25) # strongly related to height
root_series = height_series * np.maximum(np.random.normal(scale = 0.2, loc = 0.
↪7, size = num_samples), 0.3) # strongly related to height
df2 = pd.DataFrame({'height': height_series, 'width': width_series, 'root':␣
↪root_series})
minroot = root_series.min()
maxroot = root_series.max()
maxbubble = 300
```

```
ax = sns.scatterplot(data = df2, x = "height", y = "width", size = "root",␣
↪sizes = (minroot / maxroot * maxbubble, maxbubble))
ax.set_xlabel("tree height")
ax.set_ylabel("width of branches")
```

[7]: Text(0, 0.5, 'width of branches')



2.0.3 Principal Component Analysis

Principal Component Analysis can be thought of as a way to use one (or a few) linear combinations of the coordinates in order to explain all of the coordinates. It is most useful when there are strong correlations between some or all of the coordinates. Principal Component Analysis can show us which linear combinations are most (and least) useful for explaining the data.

With the trees data, the three values (height, width, and root) are highly correlated. For this reason, Principal Component Analysis shows that most of the variance can be explained by a single value which is a linear combination of height, width, and root. What this means is that if the first components_ vector in the code below (the "first principal component") is [a, b, c], for some specific numbers a, b, and c, then the value v = a * (height - mu_h) + b * (width - mu_w) + c * (root - mu_r), where the mu's are the means, is highly informative about the values of height, width, and root. In this case, because height, width, and root are so highly correlated, we can almost assume that all of the data is located along an axis defined by the vector [a, b, c]. The

point v * [a, b, c] + (mean vector) = (a * (height - mu_h) + b * (width - mu_w) + c * (root - mu_r)) * [a, b, c] + (mean vector) is therefore approximately equal to the data point (height, width, root) in question. The mean vector is [mu_h, mu_w, mu_r]. Importantly, there is still variance unexplained by this one value - just knowing v * [a, b, c] does not tell you the height, width, and root with 100% accuracy. If the second principal component is [d, e, f], then we could approximate the tree vector even more accurately as: (height, width, root) = (a * (height - mu_h) + b * (width - mu_w) + c * (root - mu_r)) * [a, b, c] + (d * (height - mu_h) + e * (width - mu_w) + f * (root - mu_r)) * [d, e, f] + (mean vector).

The printing of "components_", below, shows the first principal component [a, b, c]. If you draw a vector in the direction shown, you will be heading in the direction where the values height, width, and root change most rapidly. A later computation shows all three principal component vectors.

Optional exercise for the reader: show that if all three principal components are included, then equality is exact. This is going to be true simply because the principal components are orthonormal: the projections onto a complete set of orthonormal vectors always add up to the original vector.

(height, width, root) = (a * (height - mu_h) + b * (width - mu_w) + c * (root - mu_r)) * [a, b, c] + (d * (height - mu_h) + e * (width - mu_w) + f * (root - mu_r)) * [d, e, f] + (g * (height - mu_h) + h * (width - mu_w) + i * (root - mu_r)) * [g, h, i] + [mu_h, mu_w, mu_r].

For this proof, we must use the fact that the principal component vectors are orthonormal. That is, any pair of them are orthogonal, even with a transpose (ad + be + cf = 0, ag + bh + ci = 0, dg + eh + fi = 0, ab + de + gh = 0, ac + df + gi = 0, bc + ef + hi = 0) and any one of them is a unit vector, even with a transpose (a^2 + b^c + c^2 = 1, d^2 + e^2 + f^2 = 1, g^2 + h^2 + i^2 = 1, a^2 + d^2 + g^2 = 1, b^2 + e^2 + h^2 = 1, c^2 + f^2 + i^2 = 1). If you write out the equation above as a single vector, then apply the orthonormality rules, you should be able to prove equality.

[8]:
```
pca = decomposition.PCA(n_components = 1)
pca.fit(df2)
print(f"The first principal component explains {pca.
 ↪explained_variance_ratio_[0]} of the variance")
print(f"The components_ vector is [a, b, c] = {pca.components_}")
mu_h, mu_w, mu_r = df2.mean().values
print(f"(height, width, root) is approximately equal to ({pca.components_[0][0]:
 ↪.2} * (height - {mu_h:.2}) + {pca.components_[0][1]:.2} * (width - {mu_w:.
 ↪2}) + {pca.components_[0][2]:.2} * (root - {mu_r:.2})) * ({pca.
 ↪components_[0][0]:.2}, {pca.components_[0][1]:.2}, {pca.components_[0][2]:.
 ↪2}) + ({mu_h:.2}, {mu_w:.2}, {mu_r:.2})")
print("Examples: (estimating the the vector equals its projection onto the␣
 ↪first principal component)")
df_example = df2.copy()
df_example["v"] = pca.components_[0][0] * (df2["height"] - mu_h) + pca.
 ↪components_[0][1] * (df2["width"] - mu_w) + pca.components_[0][2] *␣
 ↪(df2["root"] - mu_r)
df_example["height_est"] = df_example["v"] * pca.components_[0][0] + mu_h
df_example["width_est"] = df_example["v"] * pca.components_[0][1] + mu_w
df_example["root_est"] = df_example["v"] * pca.components_[0][2] + mu_r
```

```
print(df_example.iloc[0:5])
print("Means:")
print(f"mu_h = {mu_h:.3}", f"mu_w = {mu_w:.3}", f"mu_r = {mu_r:.3}")
```

```
The first principal component explains 0.8686108480944689 of the variance
The components_ vector is [a, b, c] = [[0.59031629 0.71908728 0.36666083]]
(height, width, root) is approximately equal to (0.59 * (height - 5.6e+01) +
0.72 * (width - 5.9e+01) + 0.37 * (root - 3.6e+01)) * (0.59, 0.72, 0.37) +
(5.6e+01, 5.9e+01, 3.6e+01)
Examples: (estimating the the vector equals its projection onto the first
principal component)
   height       width       root          v  height_est  width_est   root_est
0      54   72.139334  31.517932   6.287751   60.001762  63.563941  38.673095
1      57   49.169527  33.768693  -7.633330   51.783921  53.553468  33.568780
2      74   83.443733  28.770850  25.215680   71.175227  77.174773  45.613225
3      77   76.353866  53.061241  30.794730   74.468631  81.186597  47.658844
4      77  111.095365  26.006307  45.856916   83.360084  92.017624  53.181557
Means:
mu_h = 56.3 mu_w = 59.0 mu_r = 36.4
```

If we were to choose a second or third component to preserve, that values below show that each would explain a small, additional amount of the variance. This means that with this particular dataset, we should likely do PCA using only one component and not two. If the components' explained variances were closer in magnitude to each other, we might have to include two or even all three values to explain the data.

[9]:
```
pca3 = decomposition.PCA(n_components = 3)
pca3.fit(df2)
print(pca3.explained_variance_ratio_)
print(pca3.components_)
```

```
[0.86861085 0.09414617 0.03724298]
[[ 0.59031629  0.71908728  0.36666083]
 [ 0.32633701 -0.62809127  0.70640322]
 [ 0.73826204 -0.29734633 -0.6054373 ]]
```

The principal component vectors shown are orthogonal to each other (their dot product is basically zero). Here, we check that using the first two vectors.

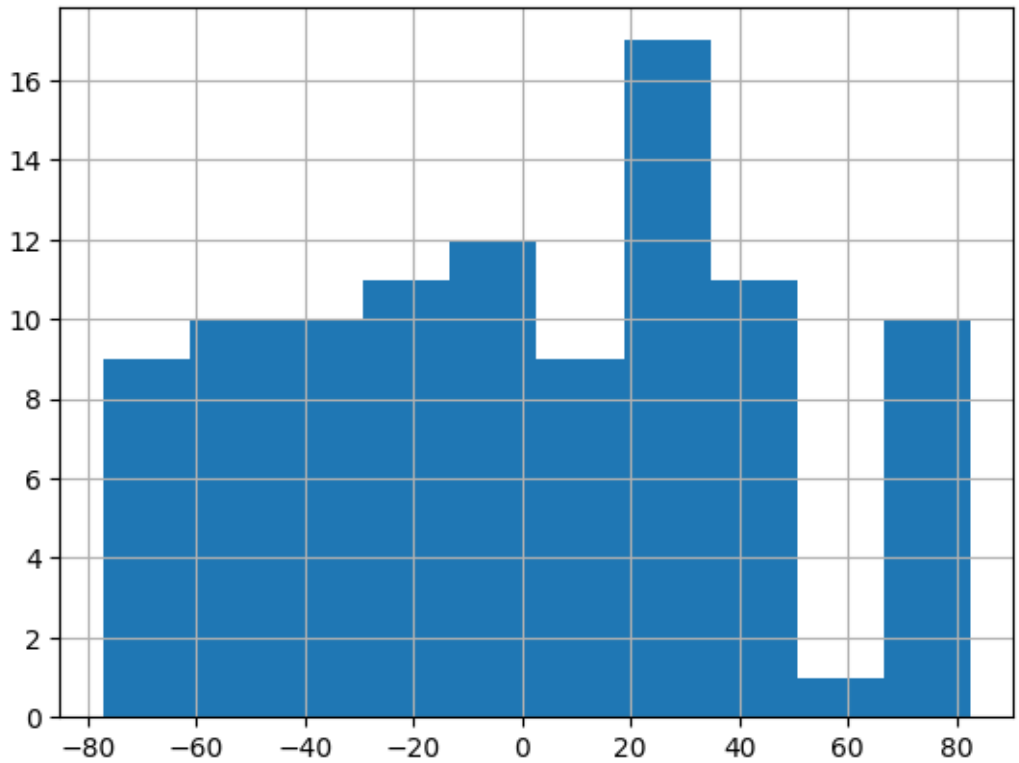
[10]:
```
print(np.array(pca3.components_[0]).dot(np.array(pca3.components_[1])))
```

```
-7.283758942514274e-17
```

We can use the projection onto the first principal component as if it were a new feature. For instance, we could plot a histogram of the data against the magnitude of the projection onto the first principal component. In fact, for some purposes, this magnitude (which we have called v) might make a better feature for data analysis than either height, width, or root. We must subtract off the mean before the dot product; the understanding is that this represents the component of the data relative to the mean.

[11]:
```
((df2 - df2.mean()).dot(np.array(pca.components_[0]))).hist()
```

[11]: <Axes: >



[12]:
```
# Showing that this is the same as the value we've called "v"
(df_example["v"] - (df2 - df2.mean()).dot(np.array(pca.components_[0]))).max()
```

[12]: 1.4210854715202004e-14

2.0.4 Linear Regression

Now let's run a linear regression model to predict the root from the height and width, as well as predicting the third principal component from the first and second. Which is easier to predict?

2.0.5 Predicting the root from the height and width

[13]:
```
linreg = linear_model.LinearRegression()
linreg.fit(df2[['height', 'width']], df2['root'])
preds = linreg.predict(df2[['height', 'width']])
metrics.mean_squared_error(preds, df2['root']) # Mean squared error as a metric
```

[13]: 138.2248735632961

[14]:
```
np.corrcoef(preds, df2['root']) # Correlation coefficient as a metric
```

[14]:
```
array([[1.        , 0.79468295],
       [0.79468295, 1.        ]])
```

2.0.6 Predicting the third principal component from the first and second

[15]:
```
first_principal_component_score = (df2 - df2.mean()).dot(np.array(pca3.
 ↪components_[0]))
second_principal_component_score = (df2 - df2.mean()).dot(np.array(pca3.
 ↪components_[1]))
third_principal_component_score = (df2 - df2.mean()).dot(np.array(pca3.
 ↪components_[2]))
```

[16]:
```
linreg_first = linear_model.LinearRegression()
pc_X = np.array([first_principal_component_score,␣
 ↪second_principal_component_score]).T
linreg_first.fit(pc_X, third_principal_component_score)
preds = linreg_first.predict(pc_X)
metrics.mean_squared_error(preds, third_principal_component_score) # The mean␣
 ↪squared error is a bit smaller here.
```

[16]:
```
78.74928905273173
```

[17]:
```
np.corrcoef(preds, third_principal_component_score) # More importantly, the␣
 ↪correlation is basically zero
```

[17]:
```
array([[1.00000000e+00, 1.55498531e-14],
       [1.55498531e-14, 1.00000000e+00]])
```

Here, we see that the projections of the first two principal components have no ability to predict the projection of the third, when measured by correlation. This is a special property of principal components. It is intended to ensure that the each component gives new information not present in the previous components.

The mean_squared_error is misleading - it makes it look like the error is less for the second computation than the first; but the correlation coefficient is giving more useful information here. We cannot compare the mean squared error for two totally different computations, because mean squared error is highly dependent on the magnitudes of the values in question!

3 2. Weekly graph question

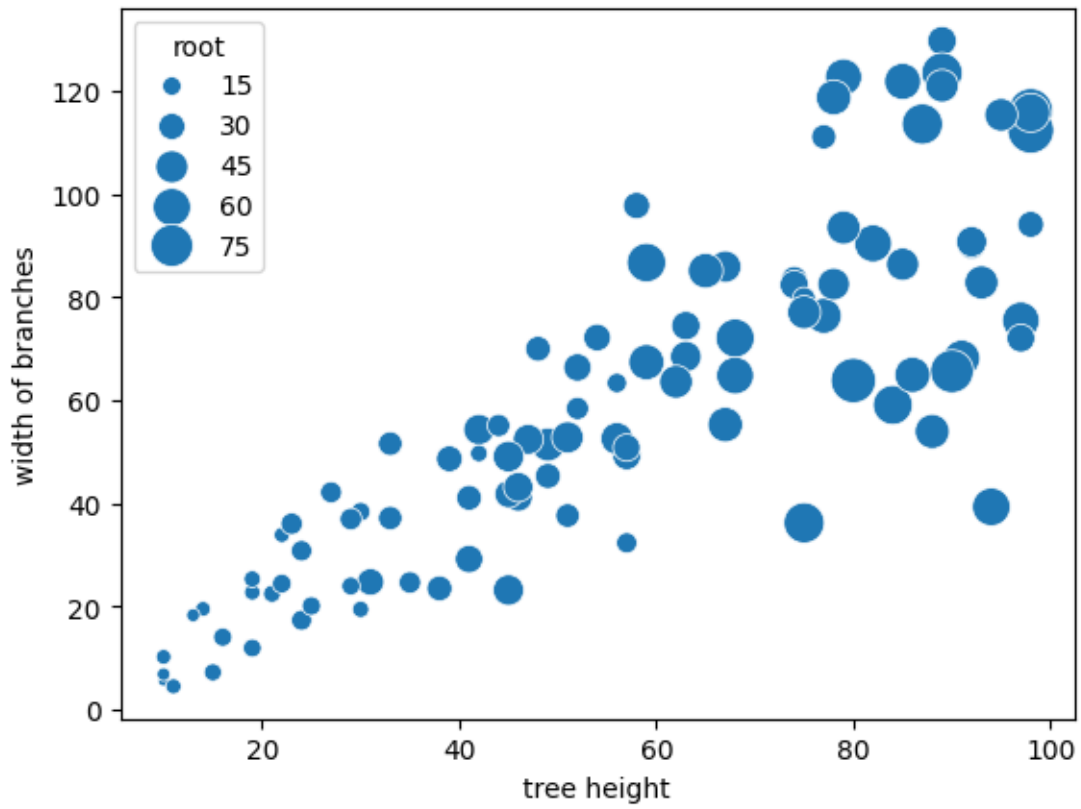
The tree data above can be graphed via either a pairplot or a bubble plot. What are the advantages and disadvantages of each?

The main advantage of a bubble plot is that one can easily see the relation of 3 variables at once, while a pair plot compares two variables at a time. On the other hand a bubble plot does not visualize the entire dataset at once, the pair plot can do this. In a pair plot one can immediately see which features have what type of relationship, even if one did not expect such a relationship.

The same type of visualization might be possible with a bubble plot, but would require many more combinations of variables.

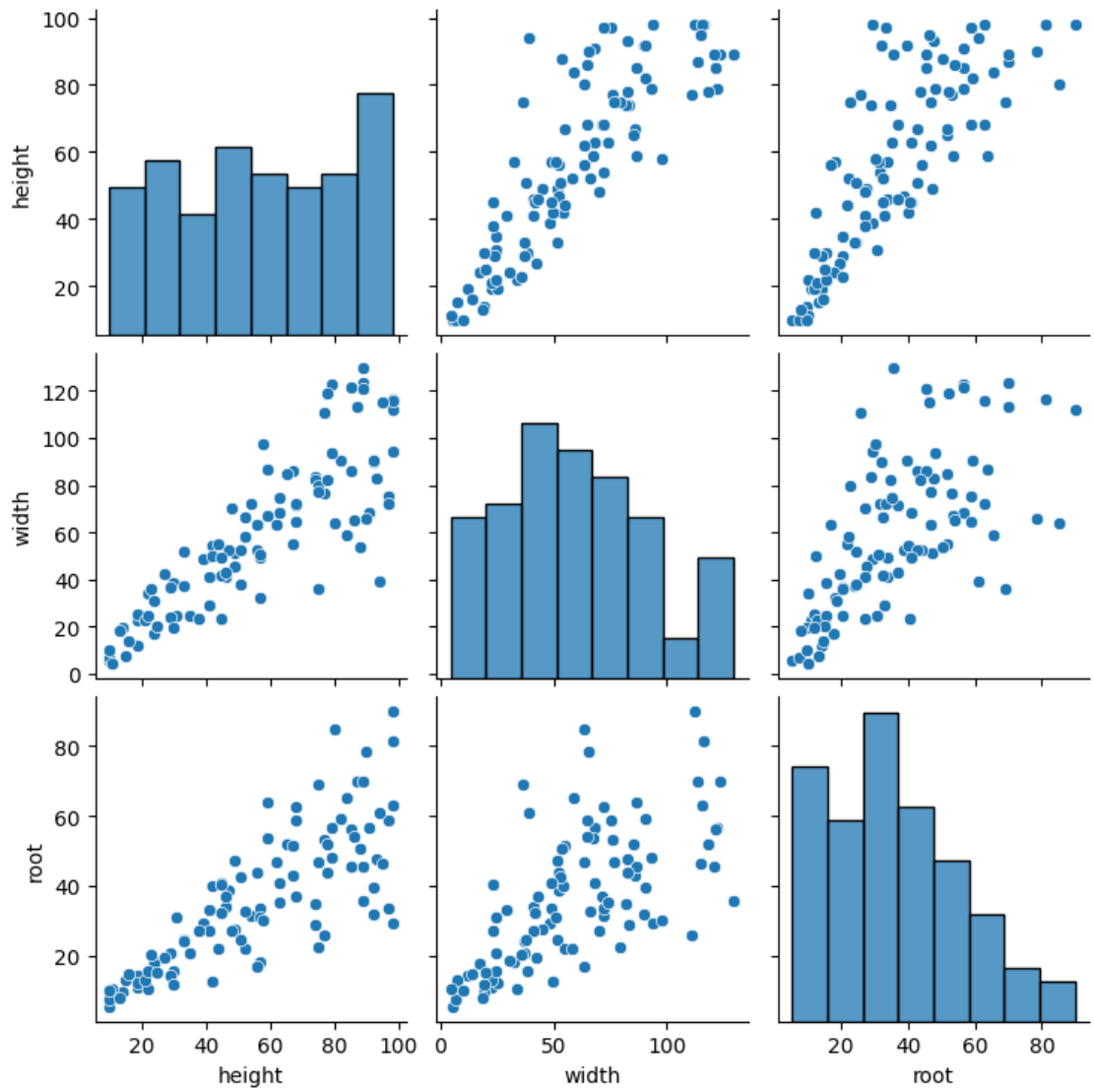
[18]:
```
ax = sns.scatterplot(data = df2, x = "height", y = "width", size = "root",␣
↪sizes = (minroot / maxroot * maxbubble, maxbubble))
ax.set_xlabel("tree height")
ax.set_ylabel("width of branches")
```

[18]: Text(0, 0.5, 'width of branches')



[19]:
```
sns.pairplot(df2)
```

[19]: <seaborn.axisgrid.PairGrid at 0x2372c878ad0>



4 3. Homework - Grocery Store Correlation

In a particular state, the price of milk is highly correlated with the price of cheese. Neither is correlated with the price of bread.

[20]:
```
num_points = 1000
np.random.seed(0)
milk_series = np.maximum(np.random.normal(scale = 0.5, loc = 2.00, size =␣
↪num_points), 1)
cheese_series = milk_series * np.maximum(np.random.normal(scale = 0.1, loc = 1.
↪00, size = num_points), 0.5)
```

```
bread_series = np.maximum(np.random.normal(scale = 1.0, loc = 3.00, size =␣
↪num_points), 1)
df3 = pd.DataFrame({"milk": milk_series, "cheese": cheese_series, "bread":␣
↪bread_series})
df3.to_csv("df3.csv", index = False)
```

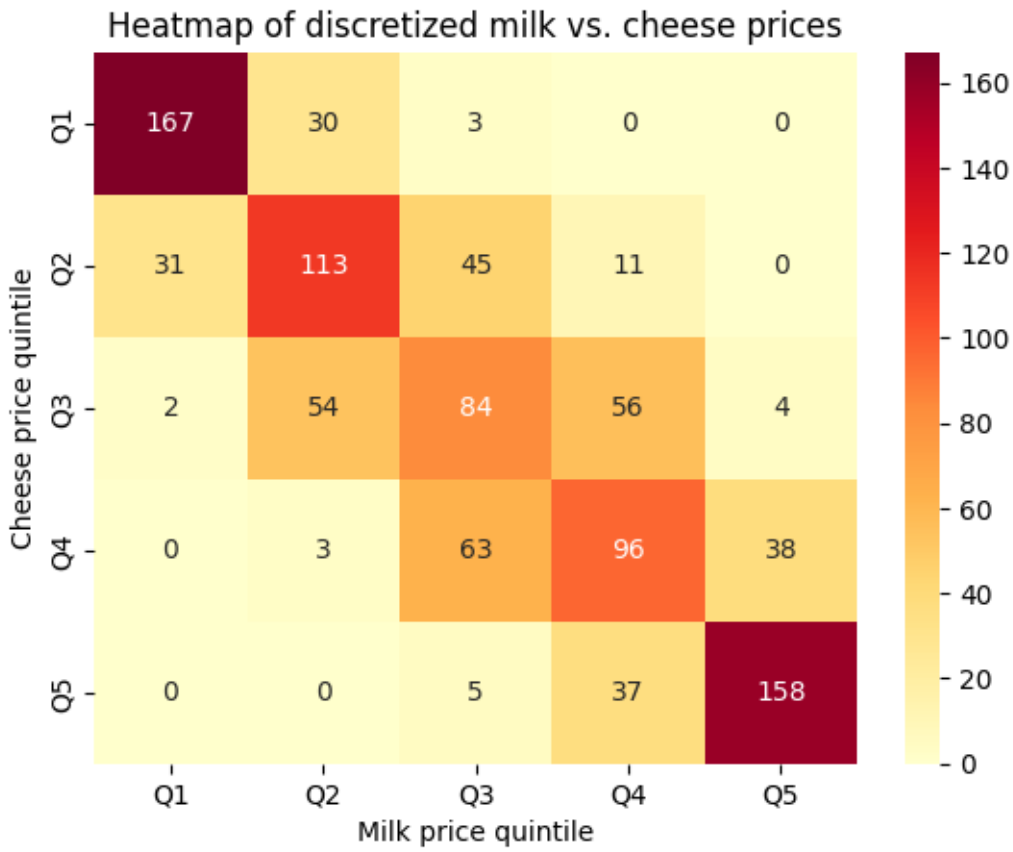
4.0.1 Heatmap

Draw a heatmap where the x-axis category is a discretized version of the milk price and the y-axis category is a discretized version of the cheese price. That is, you can choose "low", "medium", and "high" for the milk price, or, if you prefer, you can choose 1, 2, 3, 4, and 5 for the five quintiles. (The first quintile should include the bottom fifth of the data, the second quintile contains the next fifth, and so on).

Try Googling "how do I change numerical to categorical data" to get advice on this.

[21]:
```
# Discretize milk and cheese into quintiles (Q1=lowest prices, Q5=highest␣
↪prices)
df3["milk_quintile"] = pd.qcut(df3["milk"], q = 5, labels = ["Q1", "Q2", "Q3",␣
↪"Q4", "Q5"], duplicates = "drop")
df3["cheese_quintile"] = pd.qcut(df3["cheese"], q = 5, labels = ["Q1", "Q2",␣
↪"Q3", "Q4", "Q5"], duplicates = "drop")

# Count observations in each (cheese quintile, milk quintile) pair
heatmap_counts = pd.crosstab(df3["cheese_quintile"], df3["milk_quintile"])

ax = sns.heatmap(heatmap_counts, annot = True, fmt = "d", cmap = "YlOrRd")
ax.set_xlabel("Milk price quintile")
ax.set_ylabel("Cheese price quintile")
ax.set_title("Heatmap of discretized milk vs. cheese prices")
```

[21]:
```
Text(0.5, 1.0, 'Heatmap of discretized milk vs. cheese prices')
```

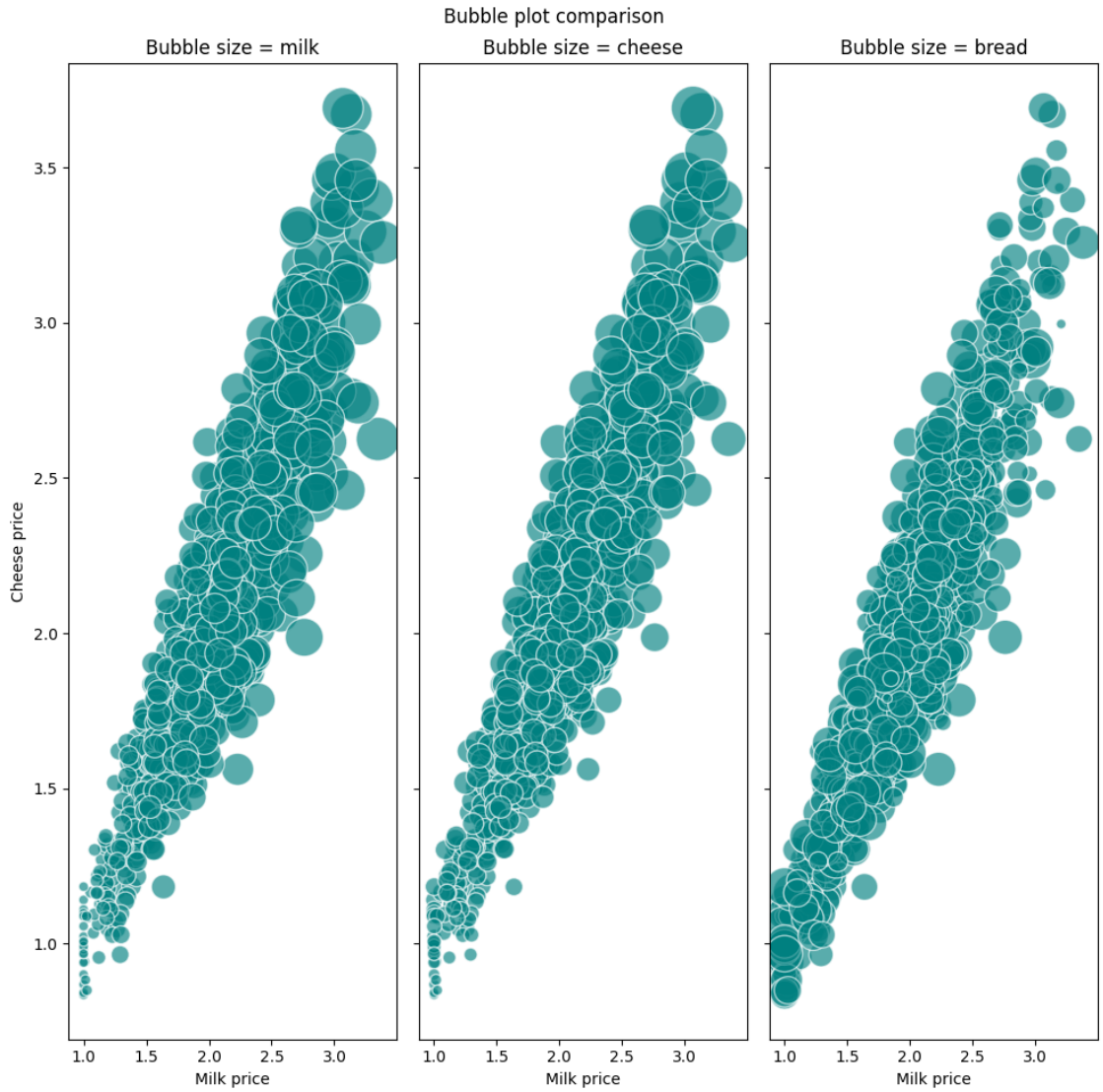


4.0.2 Bubble plot

Try all three possible bubble plots: one where the size of the dot is determined by the milk price, one where it's the cheese price, and one where it's the bread price. Which of these plots is / are the most informative and useful?

[24]:
```
# Bubble plots with x=milk, y=cheese, and bubble size based on each candidate␣
↪variable
size_columns = ["milk", "cheese", "bread"]
fig, axes = plt.subplots(1, 3, figsize = (10, 10), sharex = True, sharey = True)

for ax, size_col in zip(axes, size_columns):
    sns.scatterplot(
        data = df3,
        x = "milk",
        y = "cheese",
        size = size_col,
        sizes = (40, 800),
        alpha = 0.65,
        color = "teal",
```

```
        legend = False,
        ax = ax
    )
    ax.set_title(f"Bubble size = {size_col}")
    ax.set_xlabel("Milk price")
    ax.set_ylabel("Cheese price")

fig.suptitle("Bubble plot comparison")
plt.tight_layout()
plt.show()

print("Most informative plot: bubble size = bread.")
print("Reason: milk and cheese are already encoded on the axes, so using either␣
 ↪as bubble size is mostly redundant.")
print("Using bread as bubble size adds a third distinct variable, making the␣
 ↪chart more useful.")
```




```
Most informative plot: bubble size = bread.
Reason: milk and cheese are already encoded on the axes, so using either as
bubble size is mostly redundant.
Using bread as bubble size adds a third distinct variable, making the chart more
useful.
```

4.0.3 Principal component analysis

Perform a principal component analysis of the data. What are the first, second, and third component vectors?

Challenge question: explain why the three vectors point as they do. (Could someone have guessed the order of the three vectors, even before running the analysis?)

[ ]:
```python
# PCA on the three price variables
X = df3[["milk", "cheese", "bread"]]

# Standardize so each variable contributes comparably - note that this took a␣
↪little bit of trial and error to get right, since the default pandas std␣
↪uses ddof=1, but PCA seems to expect ddof=0
X_std = (X - X.mean()) / X.std(ddof = 0)

pca3 = decomposition.PCA(n_components = 3)
pca3.fit(X_std)

component_vectors = pd.DataFrame(
    pca3.components_,
    columns = X.columns,
    index = ["PC1", "PC2", "PC3"]
)

print("Principal component vectors (loadings):")
print(component_vectors.round(4))
print()
print("Explained variance ratio:")
print(pd.Series(pca3.explained_variance_ratio_, index = ["PC1", "PC2", "PC3"]).
↪round(4))
print()
print("Interpretation:")
print("- PC1 points mostly along the shared milk-cheese direction (both␣
↪positive and large),")
print("  because milk and cheese are strongly correlated in the data generation␣
↪process.")
print("- PC2 is mostly a bread-vs-(milk,cheese) contrast, since bread is␣
↪largely independent")
print("  from the milk-cheese pair.")
print("- PC3 captures the remaining small difference between milk and cheese.")
print("So yes: before running PCA, one could guess PC1 would align with␣
↪milk+cheese,")
print("and another component would involve bread as a separate direction.")
```

```
Principal component vectors (loadings):
       milk  cheese   bread
PC1  0.7063  0.7062 -0.0497
PC2  0.0322  0.0381  0.9988
PC3  0.7072 -0.7070  0.0042

Explained variance ratio:
PC1    0.6418
PC2    0.3326
PC3    0.0256
```

```
dtype: float64

Interpretation:
- PC1 points mostly along the shared milk-cheese direction (both positive and
large),
  because milk and cheese are strongly correlated in the data generation
process.
- PC2 is mostly a bread-vs-(milk,cheese) contrast, since bread is largely
independent
  from the milk-cheese pair.
- PC3 captures the remaining small difference between milk and cheese.
So yes: before running PCA, one could guess PC1 would align with milk+cheese,
and another component would involve bread as a separate direction.
```

4.0.4 Linear regression

Use linear regression to predict the milk price from the other two, the cheese price from the other two, and the bread price from the other two. Which prediction(s) work the best, and why?

[26]:
```python
from sklearn.model_selection import train_test_split

results = []
targets = ["milk", "cheese", "bread"]

for target in targets:
    predictors = [c for c in targets if c != target]
    X = df3[predictors]
    y = df3[target]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size = 0.25, random_state = 42
    )

    model = linear_model.LinearRegression()
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    r2 = metrics.r2_score(y_test, y_pred)
    mae = metrics.mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(metrics.mean_squared_error(y_test, y_pred))

    results.append({
        "target": target,
        "predictors": ", ".join(predictors),
        "R2": r2,
        "MAE": mae,
        "RMSE": rmse
    })
```

```
results_df = pd.DataFrame(results).sort_values(by = "R2", ascending = False).
 ↪reset_index(drop = True)
print("Regression performance (test set):")
print(results_df.round(4))
print()

best_target = results_df.loc[0, "target"]
print(f"Best prediction: {best_target}")
print("Why: cheese and milk are strongly correlated, while bread is mostly␣
 ↪independent,")
print("so models predicting milk or cheese from each other perform best and␣
 ↪bread is hardest to predict.")
```

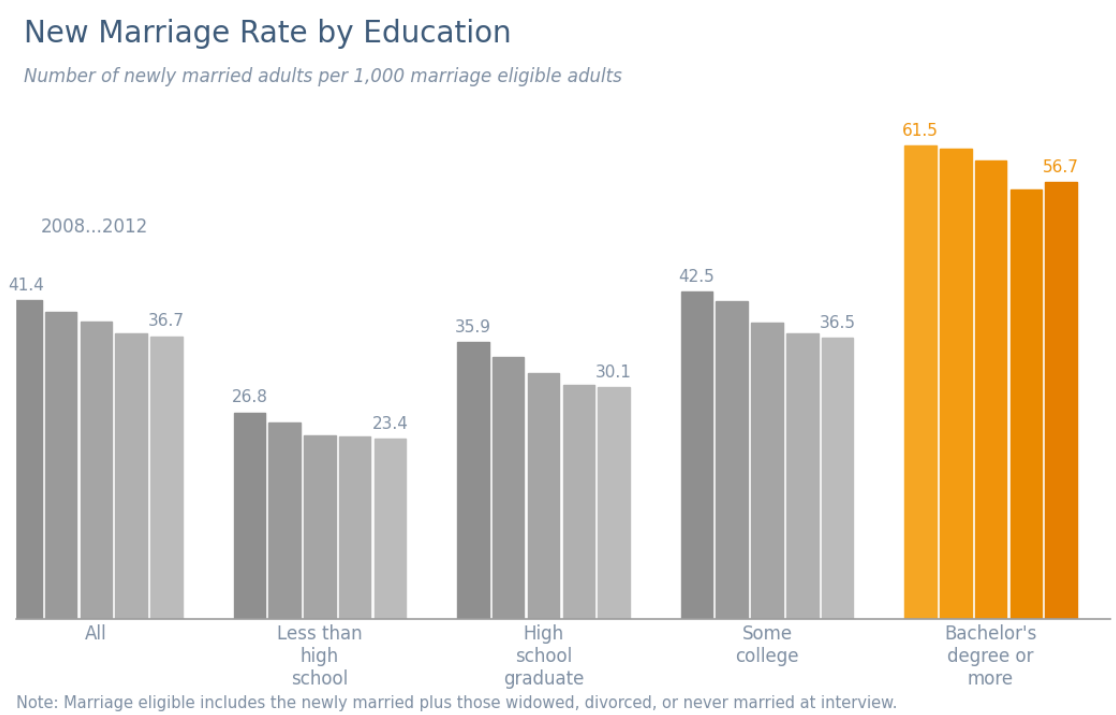
```
Regression performance (test set):
   target     predictors      R2     MAE    RMSE
0    milk  cheese, bread  0.8374  0.1498  0.1993
1  cheese    milk, bread  0.8365  0.1577  0.2107
2   bread   milk, cheese -0.0233  0.8099  0.9886

Best prediction: milk
Why: cheese and milk are strongly correlated, while bread is mostly independent,
so models predicting milk or cheese from each other perform best and bread is
hardest to predict.
```

5 4. Storytelling With Data plot

Reproduce any graph of your choice in p. 120-135 of the Storytelling With Data book as best you can. ("Use color consistently" in chapter four up to "Create a clear visual hierarchy of information" in chapter five.) You do not have to get the exact data values right, just the overall look and feel.

Reproduction of Figure 5.3 on page 131

[27]:
```
import matplotlib.pyplot as plt

categories = [
    "All",
    "Less than\nhigh\nschool",
    "High\nschool\ngraduate",
    "Some\ncollege",
    "Bachelor's\ndegree or\nmore",
]

# Five annual values (2008..2012) per education group, shaped to match the␣
 ↪figure.
data = {
    "All": [41.4, 39.8, 38.6, 37.1, 36.7],
```

```
    "Less than\nhigh\nschool": [26.8, 25.5, 23.8, 23.6, 23.4],
    "High\nschool\ngraduate": [35.9, 34.0, 31.9, 30.3, 30.1],
    "Some\ncollege": [42.5, 41.2, 38.4, 37.0, 36.5],
    "Bachelor's\ndegree or\nmore": [61.5, 61.1, 59.6, 55.8, 56.7],
}

gray_palette = ["#8f8f8f", "#9a9a9a", "#a5a5a5", "#b0b0b0", "#bbbbbb"]
orange_palette = ["#f5a623", "#f39c12", "#f0930a", "#ea8a00", "#e57f00"]

n_groups = len(categories)
n_bars = 5
group_gap = 1.4
bar_width = 0.22

fig, ax = plt.subplots(figsize=(10.6, 6.8))

for gi, cat in enumerate(categories):
    center = gi * group_gap
    offsets = (np.arange(n_bars) - (n_bars - 1) / 2) * bar_width
    values = data[cat]
    colors = orange_palette if cat == "Bachelor's\ndegree or\nmore" else␣
 ↪gray_palette

    for bi, (off, val) in enumerate(zip(offsets, values)):
        ax.bar(center + off, val, width=bar_width * 0.9, color=colors[bi],␣
 ↪edgecolor=colors[bi])

# Value labels (first and last year in each group)
for gi, cat in enumerate(categories):
    center = gi * group_gap
    vals = data[cat]
    left_x = center - 2 * bar_width
    right_x = center + 2 * bar_width
    lbl_color = "#f0930a" if cat == "Bachelor's\ndegree or\nmore" else "#7f8fa3"

    ax.text(left_x, vals[0] + 1.0, f"{vals[0]:.1f}", ha="center", va="bottom",␣
 ↪fontsize=11, color=lbl_color)
    ax.text(right_x, vals[-1] + 1.0, f"{vals[-1]:.1f}", ha="center",␣
 ↪va="bottom", fontsize=11, color=lbl_color)

# Axes styling to mirror SWD style.
ax.set_ylim(0, 70)
ax.set_yticks([])
ax.set_xticks([i * group_gap for i in range(n_groups)])
ax.set_xticklabels(categories, fontsize=12, color="#7f8fa3")
ax.tick_params(axis="x", length=0)
```

```python
for spine in ["left", "top", "right"]:
    ax.spines[spine].set_visible(False)
ax.spines["bottom"].set_color("#9e9e9e")
ax.spines["bottom"].set_linewidth(1.2)

# Title, subtitle, year range note, and footnote.
title_text = "New Marriage Rate by Education"
subtitle = "Number of newly married adults per 1,000 marriage eligible adults"
year_range_text = "2008...2012"
footnote = "Note: Marriage eligible includes the newly married plus those␣
↪widowed, divorced, or never married at interview."

ax.text(-0.45, 75, title_text, fontsize=20, color="#3d5a79", ha="left")
ax.text(-0.45, 69.8, subtitle, fontsize=12, color="#7f8fa3", style="italic",␣
↪ha="left")
ax.text(-0.35, 50.3, year_range_text, fontsize=12, color="#7f8fa3")
ax.text(-0.5, -11.5, footnote, fontsize=10.5, color="#7f8fa3", ha="left")

ax.set_xlim(-0.5, (n_groups - 1) * group_gap + 0.75)
plt.tight_layout()
plt.show()
```